Arrays in Java

Definition: *An array is a fixed size sequential collection of elements of same data type grouped under single variable name.*

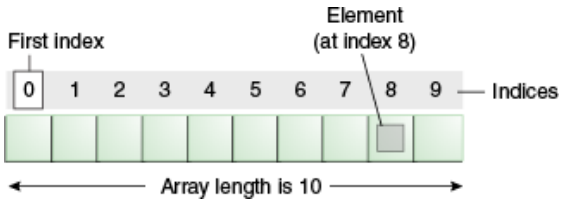
- An array is a group of like-typed variables that are referred by a common name.
- Arrays of any type can be created and may have one or more dimensions.
- A specific element in an array is accessed by its index.
- Arrays offer a convenient means of grouping related information.

`int percentage[10];`

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

- Often we need to deal with relatively large set of data.
- E.g.
  - Percentage of all the students of the college. (May be in thousands)
  - Age of all the citizens of the city. (May be lakhs)
- We need to declare thousands or lakhs of the variable to store the data which is practically not possible.
- We need a solution to store more data in a single variable.
- Array is the most appropriate way to handle such data.
- As per English Dictionary, "Array means collection or group or arrangement in a specific order."

Array declaration



Array declaration:

type var-name[];

Example:

Int student_marks[];

Above example will represent array with no value (null) To link student_marks with actual, physical array of integers, we must allocate one using new keyword.

Example:

int student_marks[] = new int[20];

- Normal Variable Declaration: int a;
- Array Variable Declaration: int b[10];
- Individual value or data stored in an array is known as an element of an array.
- Positioning / indexing of an elements in an array always starts with 0 not 1.
  - If 10 elements in an array then index is 0 to 9

Important point about Java array

- An array is **derived** datatype.
- An array is **dynamically** allocated.
- The individual elements of an array is refereed by their **index/subscript** value.
- The **subscript** for an array always begins with **0**.
- //declare and initialize and array
  int[] age = {12, 4, 5, 2, 5};
- // declare an array
  int[] age = new int[5];
- // initialize array
  age[0] = 12;
  age[1] = 4;
  age[2] = 5;

One-Dimensional Array

- An array using one subscript to represent the list of elements is called one dimensional array.
- A One-dimensional array is essentially a list of like-typed variables.
- Array declaration: type var-name[];
- Example: int student_marks[];
- Above example will represent array with no value (null).
- To link student_marks with actual array of integers, we must allocate one using new keyword.
- Example:

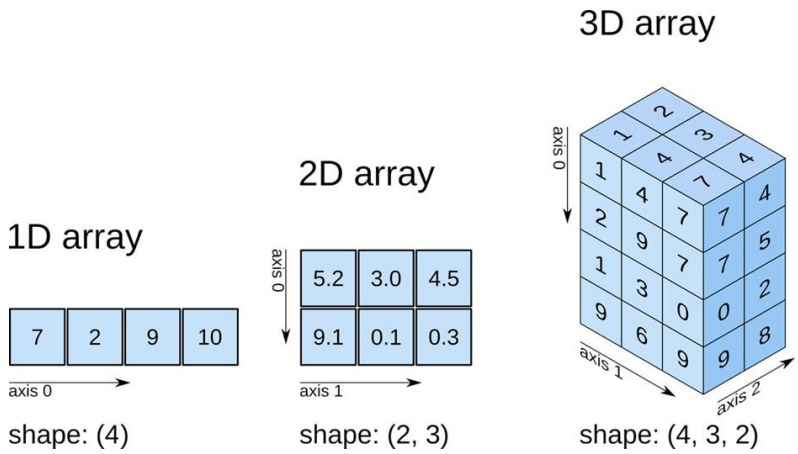
int student_marks[] = new int[20];

35	13	28	106	35	42	5	83	97	14
a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]

Example (One-Dimensional Array)

```
Class AutoArray{
      public static void main (String args[ ])
      {
            int month_days[]={31,28,31,30,31,30,31,31,
            30,31,30,31};
            System.out.println("April has" +
                        month_days[3]     + "days.");
      }  }
```

Multidimensional Array



- In java, Multidimensional arrays are actually array of arrays.
- To declare a multidimensional array variable, specify each additional index using another set of square brackets.
- For example, following declares two dimensional array:
  int twoD[ ] [ ] = new int [4] [5];
- This allocates a 4 by 5 array and assigns it to twoD. Internally this matrix is implemented as an array of arrays of int.
- Alternative Array Declaration, There is a second form that may be used to declare an array:

  type[ ] var-name;
- The square brackets follow the type specifier, and not the name of the array variable.
- For example, the following two declarations are equivalent:

  int a1[ ] = new int[4];
  int [ ] a1= new int[4];
  char twod [ ] [ ] = new char [3] [4];
  char [ ] [ ] twod = new char [3] [4];
- This alternative declaration form offers convenience when declaring several arrays at the same time. For example,

  int [ ] nums1, nums2, nums3;
- This creates 3 array variables of int type.
- Example:

  int runPerOver[][] = new int[50][6];
  Manually allocate different size:
  Int runPerOver[][] = new int[3][];
  runPerOver[0] = new int[6];
  runPerOver[1] = new int[7];
  runPerOver[2] = new int[6];

Initialization :

```
Int runPerOver[][] = {
                {0,4,2,1,0,6},
                {1,56,4,1,2,4,0},
                {6,4,1,0,2,2},
```

Example of Multidimensional Java Array

```
1. //Java Program to illustrate the use of multidimensional array
2. class Testarray3{
3. public static void main(String args[]){
4. //declaring and initializing 2D array
5. int arr[][]={{1,2,3},{2,4,5},{4,4,5}};
6. //printing 2D array
7. for(int i=0;i<3;i++){
8.  for(int j=0;j<3;j++){
9.    System.out.print(arr[i][j]+" ");
10. }
11. System.out.println();
12. }
13. }}
```

Output:

```
1 2 3
2 4 5
4 4 5
```

Initialization of an array elements

One dimensional Array

- int a[5] = { 7, 3, -5, 0, 11 }; // a[0]=7, a[1] = 3, a[2] = -5, a[3] = 0, a[4] = 11
- int a[5] = { 7, 3 }; // a[0] = 7, a[1] = 3, a[2], a[3] and a[4] are 0
- int a[5] = { 0 }; // all elements of an array are initialized to 0

Two dimensional Array

- int a[2][4] = { { 7, 3, -5, 10 }, { 11, 13, -15, 2} }; // 1st row is 7, 3, -5, 10 & 2nd row is 11, 13, -15, 2

- int a[2][4] = { 7, 3, -5, 10, 11, 13, -15, 2 }; // 1st row is 7, 3, -5, 10 & 2nd row is 11, 13, -15, 2
- int a[2][4] = { { 7, 3 }, { 11} }; // 1st row is 7, 3, 0, 0 & 2nd row is 11, 0, 0, 0
- int a[2][4] = { 7, 3 }; // 1st row is 7, 3, 0, 0 & 2nd row is 0, 0, 0, 0
- int a[2][4] = { 0 }; // 1st row is 0, 0, 0, 0 & 2nd row is 0, 0, 0, 0

Ragged Array

Ragged array is a multidimensional array where member arrays are of different size. For example, we can create a 2D array where first array is of 3 elements, and is of 4 elements. Following is the example demonstrating the concept of ragged array.

Example

```
public class Tester {
  public static void main(String[] args){
    int[][] twoDimenArray = new int[2][];
    //first row has 3 columns
    twoDimenArray[0] = new int[3];
    //second row has 4 columns
    twoDimenArray[1] = new int[4];
    int counter = 0;
    //initializing array
    for(int row=0; row < twoDimenArray.length; row++){
      for(int col=0; col < twoDimenArray[row].length; col++){
        twoDimenArray[row][col] = counter++;
      }
    }
     //printing array
    for(int row=0; row < twoDimenArray.length; row++){
      System.out.println();
      for(int col=0; col < twoDimenArray[row].length; col++){
        System.out.print(twoDimenArray[row][col] + " ");
          }
     }
  }}
```

Output

```
0 1 2
3 4 5 6
```

String Class

- In java, a string is a sequence of characters. But, unlike many other languages that implement strings as character arrays, java implements strings as objects of type String.
- In Java, **String objects are immutable**. Immutable simply means unmodifiable or unchangeable.
- Once String object is created its data or state can't be changed but a new String object is created.
- **String Constructor**
- The String class support several constructors. To create an empty String, you call the default constructor.
- For example,

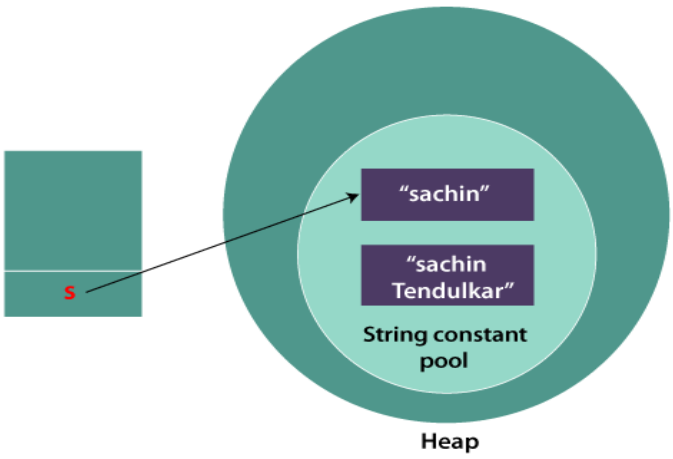
  String s=new String();
  this will create an instance of with no characters in it.
  String s = new String("Computer Deparatment");

Let's try to understand the concept of immutability by the example given below:

```
1. class Testimmutablestring{
2.  public static void main(String args[]){
3.    String s="Sachin";
4.    s.concat(" Tendulkar");//concat() method appends the string at the end
5.    System.out.println(s);//will print Sachin because strings are immutable objects
6.  }
7. }
```

Output:

```
Sachin
```



Here Sachin is not changed but a new object is created with Sachin Tendulkar. That is why String is known as immutable.

String Methods

Method Call	Task Performed
S2=s1.toLowerCase;	Conver ts the string s1 to lowercase
S2=s1.toUpperCase;	Converts the string s1 to uppercase
S2=s1.replace('x','y')	Replace all appearances of x with y.
S2=s1.trim()	Remove white spaces at the beginning and end of the string s1
S1.equals(s2)	Returns true if s1 and s2 are equal
S1.equalsIgnoreCase(s2)	Returns true if s1=s2, ignoring the case of characters
S1.length()	Gives the length of s1
S1.CharAt(n)	Gives the nth character of s1
S1.compareTo(s2)	Returns –ve if s1<s2, +ve if s1>s2, and 0 if s1=s2
S1.concat(s2)	Concatenates s1 and s2
S1.substring(n)	Gives substring starting from nth character.
S1.substring(n,m)	Gives substring starting from nth char up to mth
String.ValueOf(p)	Returns the string representation of the specified type argument.
toString()	This object (which is already a string!) is itself returned.
S1.indexOf('x')	Gives the position of the first occurrence of 'x' in the string s1
S1.indexOf('x',n)	Gives the position of 'x' that occurs after nth position in the string s1
String.ValueOf(variable)	Converts the parameter value of string representation

StringBuffer Class

- The java.lang.StringBuffer class is a thread-safe, mutable sequence of characters.
- **StringBuffer objects are Mutable, meaning that you can change the contents of the buffer without creating a new object.**
- Following are the important points about StringBuffer:
  - A string buffer is like a String, but can be modified (mutable).
  - It contains some particular sequence of characters, but the length and content of the sequence can be changed through certain method calls.
  - They are safe for use by multiple threads.
- StringBuffer defines these Constructor:
  `StringBuffer()`

```
StringBuffer(int size)
StringBuffer(String str)
StringBuffer(CharSequence chars)
```

- Remember : "StringBuffer" is mutable
- As StringBuffer class is mutable, we need not to replace the reference with a new reference as we have to do it with String class.

```
StringBuffer str1 = new StringBuffer("Hello Everyone");
str1.reverse();
// as it is mutable can not write str1 = str1.reverse();
// it will change to value of the string itself
System.out.println(str1);
// Output will be "enoyrevE olleH"
```

StringBuffer Methods

Method	Description
append(String s)	Used to append the specified string with this string.
insert(int offset, String s)	Used to insert the specified string with this string at the specified position.
replace(int startIndex, int endIndex, String str)	Used to replace the string from specified startIndex and endIndex.
delete(int startIndex, int endIndex)	Used to delete the string from specified startIndex and endIndex.
reverse()	Used to reverse the string.
Capacity()	returns the current capacity of the buffer

Difference between String and StringBuffer Class

No	String	StringBuffer
1)	String is a sequence of characters	StringBuffer is a class in Java that represents a mutable sequence of characters
2)	The String class is immutable.	The StringBuffer class is mutable.
3)	String is slow and consumes more memory when we concatenate too many strings because every time it creates new instance.	StringBuffer is fast and consumes less memory when we concatenate t strings.
4)	String class overrides the equals() method of Object class. So you can	StringBuffer class doesn't override the equals() method of Object class.

	compare the contents of two strings by equals() method.	
5)	String class is slower while performing concatenation operation.	StringBuffer class is faster while performing concatenation operation.
6)	String class uses String constant pool.	StringBuffer uses Heap memory

7)	`class Test {`
	`public static void main(String args[])`
	`{`
	`String str = "study";`
	`str.concat("tonight");`
	`System.out.println(str);  // Output: study`
	`}}`
		`class Test {`
		`public static void main(String args[])`
		`{`
		`StringBuffer strB = new StringBuffer("study");`
		`strB.append("tonight");`
		`System.out.println(strB);`
		`// Output: studytonight`
		`}}`

8)	**Methods** of String Class ✓ toLowerCase() ✓ toUpperCase() ✓ length() ✓ toString() ✓ equals() ✓ concat()	**Methods** of StringBuffer Class ✓ append() ✓ insert() ✓ replace() ✓ reverse() ✓ capacity()
9)	**Creating a StringBuffer Object** `StringBuffer sb = new StringBuffer("study");`	**Creating a String Object** `Using String Literal`

		--> String s = "Study"; Using New Keyword --> String sb = new String("study");
10)	Class **Java.lang.String**	Class **Java.lang.StringBuffer**